1. Python & Data Handling

Q1. Why did you use Pandas?
Answer: I used Pandas for loading CSV datasets, cleaning data, handling missing values, transforming columns, filtering records, grouping data and preparing DataFrames for machine-learning prediction.

Q2. Why do you convert prediction inputs into a DataFrame?
Answer: The trained Scikit-Learn model expects structured tabular input with specific feature names and ordering. Creating a DataFrame ensures the prediction input follows the expected schema.

Q3. What is the difference between loc and iloc?
Answer: loc selects data using labels, while iloc selects data using integer positions.

Q4. Why use pd.to_numeric(..., errors="coerce")?
Answer: It safely converts values to numeric format. Invalid values are converted to NaN instead of causing the application to crash.

Q5. Why is data type validation important?
Answer: Machine-learning models expect compatible data types. For example, a duration or price feature should be numerical, while categorical variables may require encoding.

---

2. Machine Learning — Very Important

Q6. Why is flight price prediction a regression problem?
Answer: Because the target variable, flight price, is continuous. The model predicts a numerical value rather than a class.

Q7. Why is customer satisfaction a classification problem?
Answer: Because the output represents categories, such as satisfied or dissatisfied.

Q8. Why did you choose Random Forest?
Answer: Random Forest is suitable for tabular data and can capture nonlinear relationships and feature interactions. It also combines multiple decision trees, which generally makes it more robust than relying on a single tree.

Q9. How does Random Forest work?
Answer: It creates multiple decision trees using randomized samples and feature selections. For regression, their predictions are aggregated, typically by averaging. For classification, the trees vote on the predicted class.

Q10. What is the difference between Random Forest Regressor and Random Forest Classifier?
Answer:

- Regressor → predicts a continuous numerical value such as ₹6,500.
- Classifier → predicts a discrete class such as satisfied/dissatisfied.

Q11. What is a decision tree?
Answer: A decision tree recursively splits data based on features to make predictions. For regression it predicts numerical values, while for classification it predicts classes.

Q12. What is overfitting?
Answer: Overfitting occurs when the model learns the training data too closely, including noise, and consequently performs poorly on unseen data.

Q13. How can you reduce overfitting in Random Forest?
Answer: By tuning parameters such as max_depth, min_samples_split, min_samples_leaf, and max_features, using cross-validation, and ensuring the training data is representative.

Q14. What is underfitting?
Answer: Underfitting occurs when the model is too simple to learn the underlying patterns in the data.

---

3. Feature Engineering — High Probability

Your application specifically contains **automatic feature engineering**, so expect questions here.

Q15. What is feature engineering?
Answer: Feature engineering is the process of transforming raw variables into features that are more useful for machine learning.

Q16. Why do you need feature engineering for flight data?
Answer: Raw flight information such as departure time, arrival time, duration and stops may not be directly in the numerical form required by the model. We transform them into model-compatible features.

Q17. Why separate hour and minute from a time value?
Answer: A machine-learning model cannot directly use a raw time string effectively. Extracting hour and minute converts the information into numerical features that can be processed by the model.

Q18. What happens if the model expects Dep_Hour but the application sends Departure_Hour?
Answer: The model or preprocessing pipeline may fail because the feature schema does not match the schema used during training.

Q19. Why is feature order important?
Answer: The model was trained using a particular feature representation. If the prediction data is incorrectly ordered or transformed, the model may receive the wrong information for each feature.

---

4. Your Model Input Error — VERY IMPORTANT

You previously encountered:

ValueError: columns missing: {'Dep_Minute', 'Dep_Hour', 'Total_Stops_Number'}

This is **exactly the type of question an evaluator could ask**.

Q20. Why did your model initially give a missing-column error?

Answer:

The trained model expected specific features such as Dep_Hour, Dep_Minute, and Total_Stops_Number, but the prediction DataFrame initially did not contain them in the expected

schema. I resolved this by inspecting the saved model's expected features and aligning the application's feature-engineering and prediction input with that schema.

Your application now includes model-input validation that displays the prediction columns, shape and data types, and also checks the saved model structure.

Q21. How did you debug the model-input problem?

Answer:

I first inspected the prediction DataFrame, including its columns, shape and data types. Then I inspected the saved model's feature_names_in_ and pipeline structure. This allowed me to identify the mismatch between the application's input schema and the model's expected schema.

The application explicitly checks feature_names_in_ and pipeline steps.

Q22. Why shouldn't you simply rename columns randomly?

Answer:

Because column names are only one part of the problem. The feature must also represent the same meaning, transformation and data type used during training. Otherwise, the model may produce an incorrect prediction even if the code runs.

This is a strong answer to remember.

---

5. Model Evaluation

Q23. What is RMSE?

Answer:

RMSE stands for Root Mean Squared Error. It measures the magnitude of prediction errors in a regression problem. Lower RMSE generally indicates better prediction performance.

Formula:

Q24. Why use RMSE for flight price prediction?

Answer:
Because flight price is a continuous target and RMSE measures the magnitude of numerical prediction errors in the same unit as the target.

Q25. What is MAE?

Answer:
Mean Absolute Error is the average absolute difference between actual and predicted values.

Q26. RMSE vs MAE?

Answer:
RMSE penalizes large errors more heavily because errors are squared. MAE treats each absolute error linearly.

---

6. Classification Questions

Q27. What is accuracy?

Answer:

Accuracy is the percentage of predictions that the classifier gets correct.

Q28. Is accuracy always sufficient?

Answer:
No. If classes are imbalanced, accuracy can be misleading. In such cases, precision, recall, F1-score and the confusion matrix should also be considered.

Q29. What is precision?

Answer:

Of all observations predicted as positive, precision tells us how many were actually positive.

Q30. What is recall?

Answer:

Of all actual positive observations, recall tells us how many were correctly identified.

Q31. What is F1-score?

Answer:
F1-score is the harmonic mean of precision and recall.

---

7. Streamlit Technical Questions

Q32. Why Streamlit instead of Flask?

Answer:

Streamlit is particularly convenient for data-science applications because I can build the UI directly in Python and integrate DataFrames, visualizations and machine-learning predictions without developing a separate frontend.

Q33. What happens when a Streamlit widget is clicked?

Answer:
Streamlit normally reruns the Python script from top to bottom. Therefore, persistent state needs to be managed appropriately.

Q34. Why did you use st.session_state?

Answer:
To preserve information such as the current page across Streamlit reruns.

Your application initializes current_page in session state and uses it for navigation.

Q35. Why are query parameters used?

Answer:
The application stores the selected page in the URL query parameter so navigation remains reliable during reruns.

Your navigate_to() function updates the query parameter and calls st.rerun().

Q36. What does st.rerun() do?

Answer:
It causes Streamlit to rerun the application script so that the new state or navigation selection is reflected immediately.

---

8. Model Deployment

Q37. Why save the model as .pkl?

Answer:
After training, the model can be serialized and saved. The Streamlit application can then load the trained model and make predictions without retraining it.

Your project uses files such as flight_price_model.pkl and customer_satisfaction_model.pkl.

Q38. Why use Joblib?

Answer:
Joblib is commonly used with Scikit-Learn objects for serialization and loading trained models efficiently.

Q39. What happens if the .pkl file is missing?

Answer:
The application cannot perform the corresponding prediction because the trained model is unavailable. The application therefore needs proper file-path validation and error handling.

Q40. Why shouldn't you train the model every time the application starts?

Answer:
Training can be computationally expensive and unnecessary for a deployed prediction application. Normally, we train and validate the model separately, save it, and load the trained artifact during inference.

---

9. Advanced Questions the Examiner May Ask

Q41. What is data leakage?

Answer:

Data leakage occurs when information that would not be available at prediction time is unintentionally used during model training. It can make evaluation results look artificially good.

Q42. What is cross-validation?

Answer:

Cross-validation repeatedly divides the training data into training and validation portions to estimate how well the model generalizes to unseen data.

Q43. What is hyperparameter tuning?

Answer:

Hyperparameters are model settings that are chosen before training, such as the number of trees or maximum depth in Random Forest. Hyperparameter tuning searches for values that improve validation performance.

Q44. What is feature importance in Random Forest?

Answer:

Feature importance estimates how much each feature contributes to the model's decision-making. It can help identify which variables are most influential.

Q45. Can Random Forest handle categorical variables directly?

Answer:

Standard Scikit-Learn Random Forest models generally require numerical inputs, so categorical variables normally need to be encoded before being passed to the model.

Q46. Why might a model perform well during testing but poorly after deployment?

Answer:

Possible reasons include data drift, differences between training and production preprocessing, unseen categories, incorrect feature engineering, leakage during training, or a mismatch between the training and inference schema.

---

10. The 10 Questions I Would Memorize First

If your viva is soon, **do not try to memorize all 46 equally**. Know these extremely well:

1. **Explain your project.**
2. **Why is flight price prediction regression?**
3. **Why is satisfaction prediction classification?**
4. **Why Random Forest?**
5. **How does Random Forest work?**
6. **What feature engineering did you perform?**
7. **What caused the missing-column error?**
8. **How did you debug the model-input mismatch?**
9. **What is RMSE and why did you use it?**
10. **How does your Streamlit application work from user input → prediction?**

⭐ **Best answer for the last one**

“The user provides flight details through the Streamlit interface. I validate the route and inputs, perform the required feature engineering, construct a DataFrame matching the trained model's expected feature schema, and pass it to the saved Random Forest Regressor. The model returns the predicted price, which I validate and display along with a fare category and prediction summary.”

That answer demonstrates **Python + data preprocessing + feature engineering + machine learning + model deployment + Streamlit**, rather than merely saying *“I built a prediction app.”* Your application explicitly performs model validation and then calls the trained model's .predict() method before displaying the result.